Nightingale — Technical Brief

Nightingale is a synthetic-data prototype of a collaborative, longitudinal patient-note clinic. This brief explains why the system is built the way it is, which engineering trade-offs were made, and what evidence supports the design. Setup and exploration instructions are in `README.md`.

1. Problem Framing & Design Goals

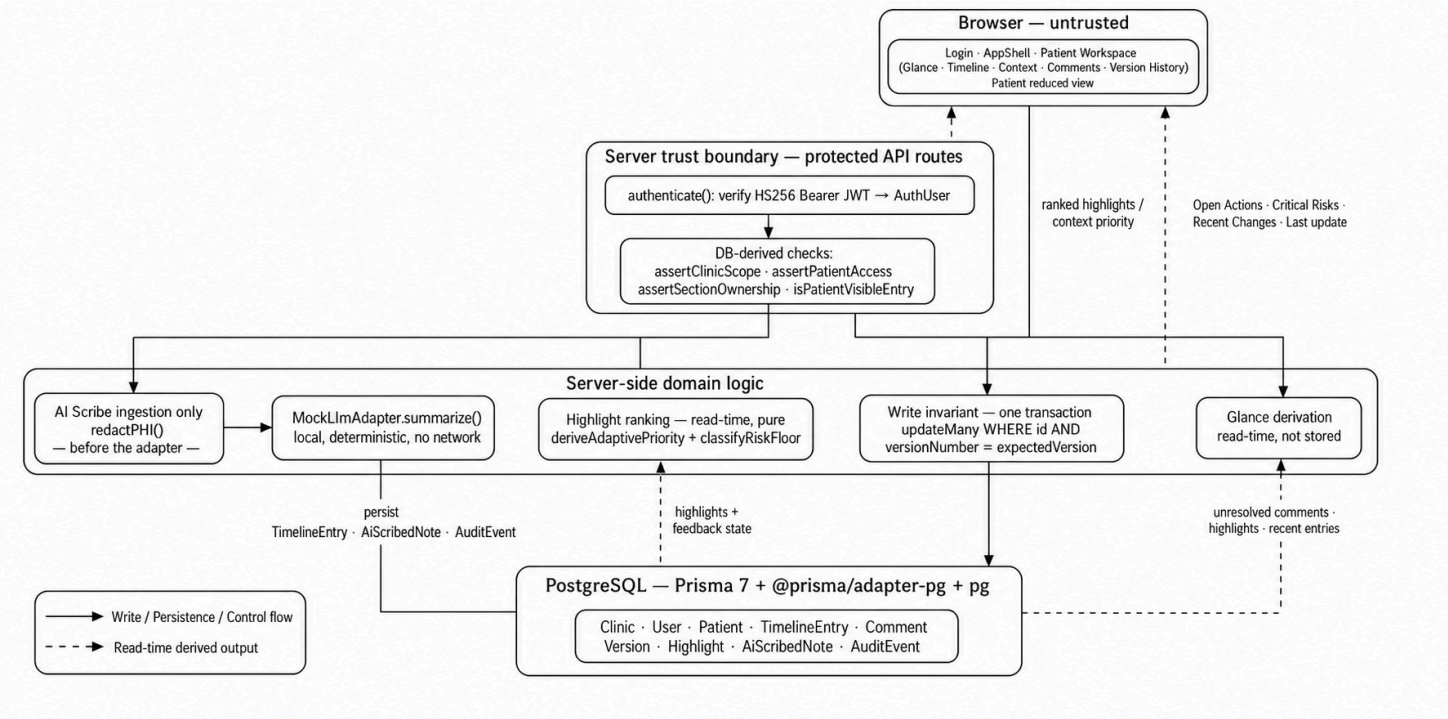
A shared longitudinal record is only useful if three properties hold together, so the design is organised around them:

1. **Access correctness.** Records cross roles and clinics, so authorization must be enforced server-side on protected API routes — derived from authenticated identity and database relationships, never from a role or id in the request body or from UI state.
2. **Change integrity and traceability.** Concurrent clinical edits must not silently overwrite each other. Content history, the action trail, and the origin of a highlighted phrase should each be reconstructable afterwards.
3. **Fast clinician orientation.** Opening a record should surface current unresolved actions, critical flags, and recent changes without re-reading the whole Timeline.

Prototype boundary. All data is synthetic; this is not a production clinical system.

2. System Architecture

Next.js (App Router) serves the UI and the JSON API under `src/app/api/**`. Protected API routes verify an HS256 bearer token, resolve an `AuthUser` (`id`, `clinicId`, `role`, optional `patientId`), and re-derive authorization from database values before any domain logic runs. Prisma 7 with the `@prisma/adapter-pg` driver adapter and the `pg` driver persists to PostgreSQL; the reference environment hosts PostgreSQL on Neon.



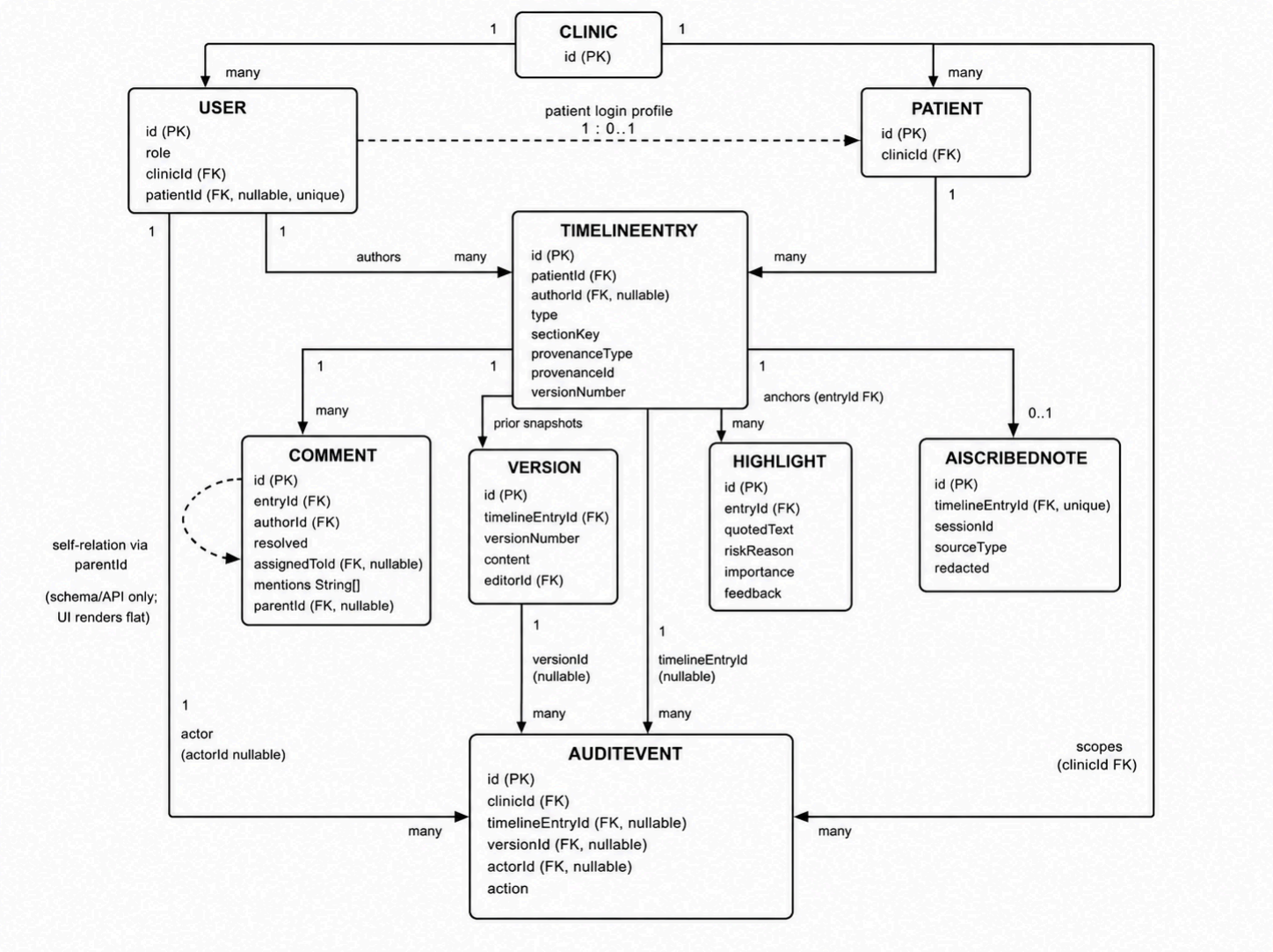
Server-side authorization. Protected API routes re-derive clinic scope, patient access, and section ownership from database values.

Derived Glance. `GET /api/patients/:id/glance` is derived per request from unresolved comments, risk-classified highlights, and recent non-`system_event` entries, avoiding a separate mutable summary store.

AI Scribe. Raw session text is redacted before it reaches the summarization adapter (§4.1).

3. Data Model & Integrity

Nine entities are persisted; provenance and adaptive ranking are represented through fields and read-time derivation, not standalone tables.



Entry-centered linkage. A `TimelineEntry` is the anchor: it carries zero or more `Comment` s, accumulates `Version` snapshots on each successful edit, may be referenced by one or more `Highlight` s (whose provenance points back to that same entry via `entryId`), and — if AI-generated — links to exactly one `AiScribedNote` . Every mutation on the entry, its versions, or its comments also produces an `AuditEvent` for traceability.

3.1 Longitudinal core. `TimelineEntry` is the per-patient chronological record. `sectionKey` controls write authorization: `staff_note → Staff` ; `plan` / `summary` / `medication → Clinician` ; null or unknown sections fail closed for every role.

3.2 Revision integrity. Writes use optimistic concurrency via `expectedVersion` . A matching version archives the replaced content as a `Version` , increments `versionNumber` , and records an audit event; a stale write returns HTTP 409 and leaves content history unchanged. `Version` therefore stores successful content snapshots, while `AuditEvent` stores mutation metadata only. Revert is forward-only: historical content becomes a new live revision rather than rewinding database state.

3.3 Provenance. A `Highlight` references one `TimelineEntry` and locates `quotedText` by exact current-text match. Provenance is same-entry only and does not validate the underlying clinical claim; AI-session provenance is stored in `provenanceType` / `provenanceId` .

4. Intelligent Behaviors: Ingestion & Prioritization

4.1 AI Scribe — Core

```
session text
  → authentication + clinic-scope check
  → redactPHI(rawText, knownNames)          # runs BEFORE the adapter
  → MockLlmAdapter.summarize(redactedText)  # local, deterministic, no network
  → transaction: TimelineEntry (authorRole = system) + AiScribedNote + AuditEvent
```

Supported session types are `doctor_consult`, `nurse_consult`, and `patient_session`, each mapped to its own AI summary and provenance type. Authentication, clinic scope, redaction, persistence, and audit are implemented; only summarization is mocked, using a deterministic local `MockLlmAdapter` with no external LLM runtime and no persisted or logged raw transcript.

4.2 Adaptive Highlight Prioritization — Bonus

Clinician feedback (`pending` / `accepted` / `rejected`) adjusts importance within ±2 after at least three reviewed examples in the same clinic; below that threshold, no adjustment is applied. A deterministic safety floor promotes highlights containing `anaphylaxis`, `chest pain`, `difficulty breathing`, or `suicidal` to `critical`, which always outranks learned adjustments. This is a trigger-based safeguard, not a comprehensive clinical risk assessment.

5. Engineering Evidence

Performance — approximation method: authenticated `GET /api/patients/:id/glance` requests were issued against a production `next start` build (Next.js 16.3.3), Clinician role, canonical synthetic dataset, on application commit `7aa5129d13958dc27174c7da0c42c4187048925b`. 10 warm-up requests were discarded; 40 warm requests measured; percentiles by nearest-rank (`rank = ceil(p × N)`). Result: **P50 = 23.01 ms, P95 = 34.23 ms**. The measured Glance endpoint met the Candidate Brief's ≤ 300 ms warm-path target in this test environment. This is authenticated server endpoint response latency, not full browser rendering or user-perceived page latency; browser/render latency was not measured. Method and raw samples: `docs/performance-evidence.md`.

Validation. Behaviour was checked with targeted Python regression tests and co-located TypeScript unit tests, route/security checks, a repository-level evidence audit, and user-performed manual browser verification of the interaction paths. The required Core micro-tests are present — `test_rbac_scope.py`, `test_revision_history.py`, `test_highlight_provenance.py`, `test_concurrent_edits.py` — with further Python and TypeScript tests covering the remaining behaviors. There is no independent automated end-to-end browser suite.

6. Assumptions, Trade-offs & Scope Boundaries

Decision	Rationale	Trade-off
Derived Glance	Avoids a second mutable summary store that could drift from the record	Read-time computation on each request
Flat comments	Simpler, reliable collaboration surface at prototype scale	`parentId` exists in schema/API, but nesting and a reply UI are not rendered
Same-entry provenance	Deterministic quote-level traceability without another source graph	No cross-entry evidence graph / `sourceEntryId`
Deterministic adaptive logic	Auditable with the low feedback volume available	Not semantic ML; no recency or feedback-history model
Local `MockLlmAdapter`	Exercises auth / redaction / persistence / provenance boundaries without an external model dependency	No production summarization inference
Forward-only revert	Preserves monotonic, auditable history	Revisions accumulate instead of the database being rewound

Scope boundaries. Mentions/assignments do not generate notifications; PHI redaction is heuristic and Singapore-oriented; authentication uses prototype-grade passwordless JWT sessions with no per-request revocation. Adaptive prioritization is implemented, while hybrid storage/data decay, voice capture, and structured clinical entity tagging are not. Tiering older immutable Timeline content to lower-cost storage remains a proposed production extension.